Dhivyuga - Comprehensive Redesign Document

Appwrite Migration: Complete Feature Specification

This document outlines every feature in the Dhivyuga system and how it maps to the Appwrite-based redesign.

---

1. Project Overview

Dhivyuga (Divine Era) is a modern spiritual search engine for discovering sacred mantras. It bridges ancient Hindu spiritual wisdom with contemporary technology, providing a searchable database of 50+ mantras with comprehensive recitation guidelines, multi-language translations, and an admin CMS.

Current Stack: Next.js 15 + Supabase (PostgreSQL) + TailwindCSS + Netlify

Target Stack: Next.js 15 + Appwrite (Cloud/Self-hosted) + TailwindCSS + Netlify

---

2. Feature Inventory

2.1 Public Features

F1: Sacred Mantra Search Engine

- **Description:** Google-like homepage with centered search bar for mantra discovery
- **Current Implementation:** PostgreSQL full-text search with tsvector and GIN index
- **Components:** SearchBar, QuickFilters, SearchFilters
- **Appwrite Approach:** Appwrite fulltext index on collections; optional Meilisearch integration for advanced ranking and typo tolerance

F2: Autocomplete Suggestions

- **Description:** Real-time search suggestions as user types (debounced 300ms)
- **Current Implementation:** ILIKE prefix matching across mantras, deities, categories tables
- **Components:** SearchBar (dropdown suggestions)
- **Appwrite Approach:** `Query.startsWith()` on title/name attributes across 3 collections

F3: Quick Filter Chips

- **Description:** 4 preset filter buttons on homepage (Wealth, Protection, Health, Wisdom)
- **Current Implementation:** Hardcoded links to `/search?category={name}`
- **Components:** QuickFilters
- **Appwrite Approach:** No backend change needed (static links)

F4: Advanced Filtering

- **Description:** Sidebar with dropdowns for category, deity, kalam, time of day, recitation count

- **Current Implementation:** API route `/api/filters` fetches from 4 tables; search applies `.eq()` filters
- **Components:** SearchFilters
- **Appwrite Approach:** `databases.listDocuments()` with `Query.equal()` filters on each attribute

F5: Trending Mantras

- **Description:** Grid showing top 6 most viewed mantras on homepage
- **Current Implementation:** `ORDER BY view_count DESC LIMIT 6` with deity/category joins
- **Components:** TrendingMantras, MantraCard
- **Appwrite Approach:** `Query.orderDesc('view_count')` + `Query.limit(6)` on mantras collection

F6: Browse by Category

- **Description:** Grid of all spiritual categories with mantra counts
- **Current Implementation:** Fetch active categories from `/api/categories`
- **Components:** CategoryGrid
- **Appwrite Approach:** `databases.listDocuments('categories')` with `Query.equal('is_active', true)`

F7: Browse by Deity

- **Description:** Grid of all deities with links to filtered search
- **Current Implementation:** Fetch active deities from `/api/deities`
- **Components:** DeityGrid
- **Appwrite Approach:** `databases.listDocuments('deities')` with `Query.equal('is_active', true)`

F8: Mantra Detail Page

- **Description:** Comprehensive mantra view with all metadata, translations, and related mantras
- **Current Implementation:** API `/api/mantras/[id]` with 7-table join + translations + related mantras
- **Components:** MantraDisplay
- **Appwrite Approach:** `databases.getDocument()` + separate queries for related collections (no automatic joins in Appwrite - requires multiple queries or denormalization)

F9: Multi-Language Translations

- **Description:** Tab-based language switcher showing mantra in English, Hindi, Tamil, Sanskrit
- **Current Implementation:** `mantra_translations` table with language FK, sorted by sort_order
- **Components:** MantraDisplay (language tabs), LanguageManager (admin)
- **Appwrite Approach:** `mantra_translations` collection with `Query.equal('mantra_id', id)`, client-side language tab switching

F10: Text-to-Speech

- **Description:** Play button that reads mantra text aloud using browser speech synthesis
- **Current Implementation:** Web Speech API (`speechSynthesis.speak()`)
- **Components:** MantraCard, MantraDisplay
- **Appwrite Approach:** No backend change (browser API)

F11: Share Mantra

- **Description:** Native share button for sharing mantra links
- **Current Implementation:** Navigator.share API

- **Components:** MantraCard, MantraDisplay
- **Appwrite Approach:** No backend change (browser API)

F12: Copy to Clipboard

- **Description:** Copy mantra text or transliteration to clipboard
- **Current Implementation:** Clipboard API
- **Components:** MantraDisplay
- **Appwrite Approach:** No backend change (browser API)

F13: View Counter

- **Description:** Tracks and displays how many times each mantra has been viewed
- **Current Implementation:** POST to `/api/mantras/[id]/view` increments view_count
- **Components:** MantraCard, MantraDisplay
- **Appwrite Approach:** Appwrite Function for atomic increment, or direct `databases.updateDocument()` with incremented value

F14: Navagraha (Planetary) Information

- **Description:** Educational page showing 9 planetary deities with properties
- **Current Implementation:** Direct Supabase query filtering deities with graha-related names
- **Components:** `/grahas` page
- **Appwrite Approach:** `Query.contains()` or tag-based filtering on deities collection

F15: Contact Form

- **Description:** Contact form with name, email, subject, message fields
- **Current Implementation:** Simulated submission (no actual backend)
- **Components:** `/contact` page
- **Appwrite Approach:** Appwrite Function to send email, or Appwrite Messaging service

F16: About Page

- **Description:** Mission, vision, features, and core values
- **Current Implementation:** Static React content
- **Appwrite Approach:** No backend change (static content)

F17: Terms of Service Page

- **Description:** Legal terms and conditions
- **Current Implementation:** Static React content
- **Appwrite Approach:** No backend change (static content)

2.2 Admin Features

A1: Admin Authentication

- **Description:** Email/password login for admin access
- **Current Implementation:** Supabase Auth (`signInWithPassword`, `signOut`, `getSession`, `onAuthStateChange`)
- **Components:** AuthContext, AuthGuard, Login page

• **Appwrite Approach:** Appwrite Account API (`account.createEmailPasswordSession()`, `account.deleteSession()`, `account.get()`)

A2: Admin Dashboard

• **Description:** Overview with 4 stat cards (Total Mantras, Total Views, Categories, Deities)

• **Current Implementation:** Aggregate queries on mantras, categories, deities tables

• **Components:** DashboardCards, DataTable

• **Appwrite Approach:** `databases.listDocuments()` with counting; Appwrite doesn't have native aggregation, so count via `.total` property of list response

A3: Mantra CRUD

• **Description:** Full create/read/update/delete for mantras with form fields: title, text, deity, category, recitation count, recitation time

• **Current Implementation:** Direct Supabase client-side CRUD with Dialog form

• **Form Fields:** title (text), text (textarea), deity_id (select), category_id (select), count_id (select), time_id (select)

• **Components:** Admin mantras page, Dialog, Tabs

• **Appwrite Approach:** `databases.createDocument()`, `updateDocument()`, `deleteDocument()`, `listDocuments()`

A4: Mantra Translation Management

• **Description:** Per-language translation editor with WYSIWYG fields

• **Current Implementation:** Full CRUD via API routes (`/api/mantras/[id]/translations`)

• **Form Fields per Language:** text (WYSIWYG), transliteration (textarea), pronunciation_guide (textarea), meaning (WYSIWYG), benefits (dynamic array), usage_notes (WYSIWYG)

• **Components:** LanguageManager, WysiwygEditor (React Quill)

• **Appwrite Approach:** CRUD on `mantra_translations` collection via `databases.*Document()` methods

A5: Category CRUD

• **Description:** Create/read/update/delete categories with mantra count display

• **Current Implementation:** Direct Supabase client-side CRUD, card grid layout with stats sidebar

• **Form Fields:** name (text, required), description (textarea), is_active (toggle)

• **Components:** Admin categories page, Dialog, motion cards

• **Appwrite Approach:** CRUD on `categories` collection

A6: Deity CRUD

• **Description:** Create/read/update/delete deities with active toggle and mantra count

• **Current Implementation:** Direct Supabase client-side CRUD, card grid with stats sidebar

• **Form Fields:** name (text, required), description (textarea), is_active (toggle)

• **Components:** Admin deities page, Dialog, motion cards

• **Appwrite Approach:** CRUD on `deities` collection

A7: Settings Management (4 entities)

• **Description:** Tabbed interface managing recitation times, counts, kalams, and time ranges

• **Current Implementation:** Dynamic table name based on active tab, unified CRUD

• **Tabs & Form Fields:**

• Recitation Times: name, description

• Recitation Counts: count_value (number), description

• Kalams: name, description, is_auspicious (checkbox)

• Time Ranges: start_time (time), end_time (time), description

• **Components:** Admin settings page, 4 Tabs, Tables, Dialog

• **Appwrite Approach:** 4 separate collections, CRUD via `databases.*Document()` methods

A8: Analytics Dashboard

• **Description:** Metrics display with total views, top mantras, category breakdown

• **Current Implementation:** Aggregate queries on mantras table (some hardcoded sample data)

• **Components:** Metric cards, top-5 list, progress bars

• **Appwrite Approach:** `listDocuments()` with `Query.orderDesc('view_count')` + `Query.limit(5)`; manual aggregation in application code

A9: Admin Setup

• **Description:** One-time initial admin user creation with default credentials

• **Current Implementation:** API POST `/api/create-admin` using Supabase Admin Auth

• **Components:** Setup page with form

• **Appwrite Approach:** Server-side Appwrite `users.create()` with Admin SDK + team assignment

A10: Admin Layout & Navigation

• **Description:** Responsive sidebar with navigation to all admin sections

• **Current Implementation:** AdminLayout component with fixed sidebar (desktop) / overlay (mobile)

• **Navigation Items:** Dashboard, Mantras, Categories, Deities, Settings, Analytics

• **Components:** AdminLayout

• **Appwrite Approach:** No backend change (pure UI component)

---

3. Data Model: Appwrite Collections Design

3.1 Database: `dhivyuga_db`

Collection: `mantras`

Attribute	Type	Required	Index
title	string(500)	Yes	fulltext
text	string(65535)	Yes	fulltext
category_id	string(36)	No	key
deity_id	string(36)	No	key
count_id	string(36)	No	key
time_id	string(36)	No	key
kalam_id	string(36)	No	key

Attribute	Type	Required	Index
range_id	string(36)	No	key
view_count	integer	No (default 0)	key (desc)

Permissions: Read: any | Create/Update/Delete: team:admins

Collection: `mantra_descriptions`

Attribute	Type	Required
mantra_id	string(36)	Yes
meaning	string(65535)	Yes
transliteration	string(65535)	No
pronunciation_guide	string(65535)	No
benefits	string[]	No
historical_context	string(65535)	No
spiritual_significance	string(65535)	No
usage_instructions	string(65535)	No
precautions	string(65535)	No
related_scriptures	string[]	No

Permissions: Read: any | Create/Update/Delete: team:admins

Collection: `mantra_translations`

Attribute	Type	Required
mantra_id	string(36)	Yes
language_id	string(36)	Yes
text	string(65535)	Yes
transliteration	string(65535)	No
pronunciation_guide	string(65535)	No
meaning	string(65535)	No
benefits	string[]	No
usage_notes	string(65535)	No

Permissions: Read: any | Create/Update/Delete: team:admins

Collection: `categories`

Attribute	Type	Required
name	string(255)	Yes
description	string(65535)	No
is_active	boolean	No (default true)

Permissions: Read: any | Create/Update/Delete: team:admins

Collection: `deities`

Attribute	Type	Required
name	string(255)	Yes
description	string(65535)	No
sanskrit_name	string(255)	No
day_of_week	string(50)	No
color	string(50)	No
gemstone	string(100)	No
metal	string(100)	No
element	string(100)	No
direction	string(50)	No
is_active	boolean	No (default true)

Permissions: Read: any | Create/Update/Delete: team:admins

Collection: `recitation_counts`

Attribute	Type	Required
count_value	integer	Yes
description	string(65535)	No

Permissions: Read: any | Create/Update/Delete: team:admins

Collection: `recitation_times`

Attribute	Type	Required
name	string(255)	Yes
description	string(65535)	No

Permissions: Read: any | Create/Update/Delete: team:admins

Collection: `kalams`

Attribute	Type	Required
name	string(255)	Yes
planet	string(100)	No
description	string(65535)	No
is_auspicious	boolean	No (default true)

Permissions: Read: any | Create/Update/Delete: team:admins

Collection: `time_ranges`

Attribute	Type	Required
start_time	string(10)	Yes
end_time	string(10)	Yes

Attribute	Type	Required
description	string(65535)	No

Permissions: Read: any | Create/Update/Delete: team:admins

Collection: `languages`

Attribute	Type	Required
code	string(10)	Yes
name	string(100)	Yes
native_name	string(100)	No
direction	string(3)	No (default 'ltr')
sort_order	integer	No (default 0)
is_active	boolean	No (default true)

Permissions: Read: any | Create/Update/Delete: team:admins

---

4. Authentication Redesign

Current (Supabase Auth)

- `supabase.auth.signInWithPassword({ email, password })`
- `supabase.auth.signOut()`
- `supabase.auth.getSession()`
- `supabase.auth.onAuthStateChange()`
- `supabase.auth.admin.createUser()` (server-side)

Target (Appwrite Auth)

- `account.createEmailPasswordSession(email, password)`
- `account.deleteSession('current')`
- `account.get()` (check current session)
- Event listener via Appwrite Realtime (optional)
- `users.create()` (server-side Admin SDK)
- Team-based authorization: create 'admins' team, check membership for protected routes

---

5. Search Strategy Redesign

Current (PostgreSQL Full-Text Search)

- tsvector column with GIN index
- `textSearch('search_vector', query)` via Supabase
- Automatic search vector updates via database triggers

- Ranking by relevance + view_count

Target (Appwrite Search)

Option A: Appwrite Native Fulltext Index

- Create fulltext indexes on mantras collection (title, text)
- Use `Query.search('title', query)` for searching
- Simpler but less powerful than PostgreSQL tsvector
- No automatic ranking by relevance

Option B: Meilisearch Integration (Recommended for production)

- Appwrite supports Meilisearch as an external search engine
- Provides typo tolerance, ranking, and faceted search
- Sync mantras collection to Meilisearch via Appwrite Function triggers
- Best experience for users but requires additional infrastructure

---

6. Migration Phases

Phase 1: Foundation (Days 1-2)

- Set up Appwrite project (Cloud or self-hosted)
- Install `appwrite` SDK
- Create `lib/appwrite.ts` replacing `lib/supabase.ts`
- Configure environment variables

Phase 2: Database (Days 3-5)

- Create all 10 collections with attributes and indexes
- Set collection permissions
- Write data migration script (Supabase export to Appwrite import)
- Seed all reference data

Phase 3: Authentication (Day 6)

- Rewrite AuthContext for Appwrite Account API
- Create admin user and 'admins' team
- Update AuthGuard component
- Update login/logout flows

Phase 4: API Routes (Days 7-10)

- Rewrite all 17 API routes to use Appwrite SDK
- Handle lack of automatic joins (multiple queries or denormalization)
- Implement search with Appwrite fulltext or Meilisearch
- Implement view counter logic

Phase 5: Admin Pages (Days 11-13)

- Update all 6 admin pages (dashboard, mantras, categories, deities, settings, analytics)
- Replace direct Supabase calls with Appwrite SDK calls
- Update LanguageManager component
- Test all CRUD operations

Phase 6: Public Pages (Days 14-15)

- Update Grahas page (direct Supabase to Appwrite)
- Verify all public components work with new API responses
- Test search, filters, trending, detail pages

Phase 7: Appwrite Functions (Days 16-17)

- View counter atomic increment function
- Contact form email function (optional)
- Search index sync function (if using Meilisearch)

Phase 8: Testing & Cleanup (Days 18-20)

- Remove all Supabase dependencies
- Delete `supabase/` directory and related scripts
- Update `package.json` (remove `@supabase/supabase-js`, add `appwrite`)
- Full end-to-end testing
- Update environment variables on Netlify
- Performance audit
- Deploy

---

7. Risk Assessment

Risk	Severity	Mitigation
Search quality degradation	High	Use Meilisearch integration or build application-level ranking
No automatic joins in Appwrite	Medium	Multiple sequential queries or denormalize data
Data loss during migration	Medium	Full Supabase backup before starting; test on staging
No database triggers in Appwrite	Medium	Use Appwrite Functions with event triggers
Appwrite query limitations	Medium	Some complex filters may need application-level processing
Auth session handling differences	Low	Appwrite session management is straightforward
Downtime during cutover	Low	Run both backends in parallel during migration

8. Files to Create/Modify

New Files

File	Purpose
`lib/appwrite.ts`	Appwrite client initialization + types
`scripts/migrate-to-appwrite.ts`	Data migration script
`scripts/seed-appwrite.ts`	Seed reference data

Files to Modify (26 Supabase touchpoints)

All files listed in the Deep Analysis document Section 7 will need Supabase references replaced with Appwrite SDK calls.

Files to Delete

File	Reason
`lib/supabase.ts`	Replaced by `lib/appwrite.ts`
`supabase/schema.sql`	No longer needed (Appwrite manages schema)
`app/api/create-admin/route.ts`	Replace with Appwrite setup script
`app/api/seed-grahas/route.ts`	Replace with Appwrite seed script

Dependencies to Change

Remove	Add
`@supabase/supabase-js`	`appwrite`
`@supabase/ssr` (if present)	-

9. Environment Variables

Remove

NEXT_PUBLIC_SUPABASE_URL

NEXT_PUBLIC_SUPABASE_ANON_KEY

SUPABASE_SERVICE_ROLE_KEY

Add

NEXT_PUBLIC_APPWRITE_ENDPOINT

NEXT_PUBLIC_APPWRITE_PROJECT_ID

APPWRITE_API_KEY (server-side only)

NEXT_PUBLIC_APPWRITE_DATABASE_ID

---

10. Success Criteria

- [ ] All 17 public features working identically to current version
- [ ] All 10 admin features working with full CRUD
- [ ] Search returns relevant results with acceptable speed (< 500ms)
- [ ] Admin authentication working with Appwrite
- [ ] View counter incrementing correctly
- [ ] Multi-language translations rendering properly
- [ ] All 10 collections populated with migrated data
- [ ] Zero Supabase dependencies remaining in codebase
- [ ] Netlify deployment successful with Appwrite backend
- [ ] Performance audit passing (Lighthouse score > 80)

Document generated: April 15, 2026

Project: Dhivyuga - Sacred Mantra Search Engine

Migration Target: Supabase to Appwrite